MicroVM Grader

A LeetCode-style code grader that runs untrusted Python inside throwaway AWS Lambda MicroVMs. Built for a recorded conference demo. Region ap-southeast-1.

What it does

- You submit Python for the **two-sum** problem.
- Every submission runs in its **own fresh MicroVM**, restored from a snapshot of the `grader-image` image.
- The grader scores it (5 tests), returns a verdict and timings, and **always terminates** the VM.
- Seven hostile payloads (loop, fork bomb, memory bomb, `rm -rf`, network exfil, answer theft, runner kill) are contained. A correct solution run before and after proves the platform is unaffected.

The key idea: two trust zones

- **Trusted**: the grader (CLI or Lambda). Holds the expected answers, AWS credentials and the comparison logic.
- **Untrusted**: the MicroVM. Receives only `code` and `inputs`. Has no AWS role, no answers, no network.

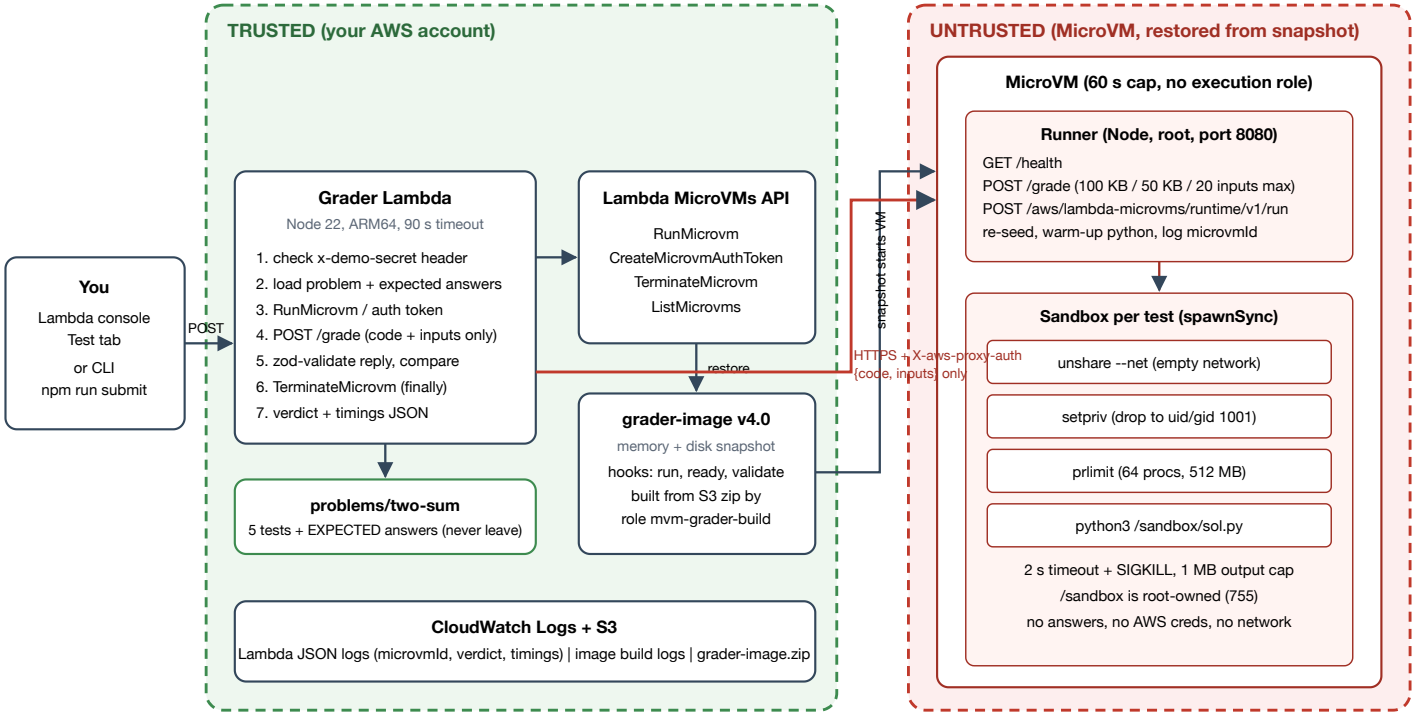
Verdicts

Verdict	Meaning
ACCEPTED	all tests pass
WRONG_ANSWER	ran fine, some output wrong
TIMEOUT	a test exceeded 2 s
KILLED	process died on a signal
ERROR	non-zero exit (e.g. permission denied, no network)

Deployed resources (account 498556006298)

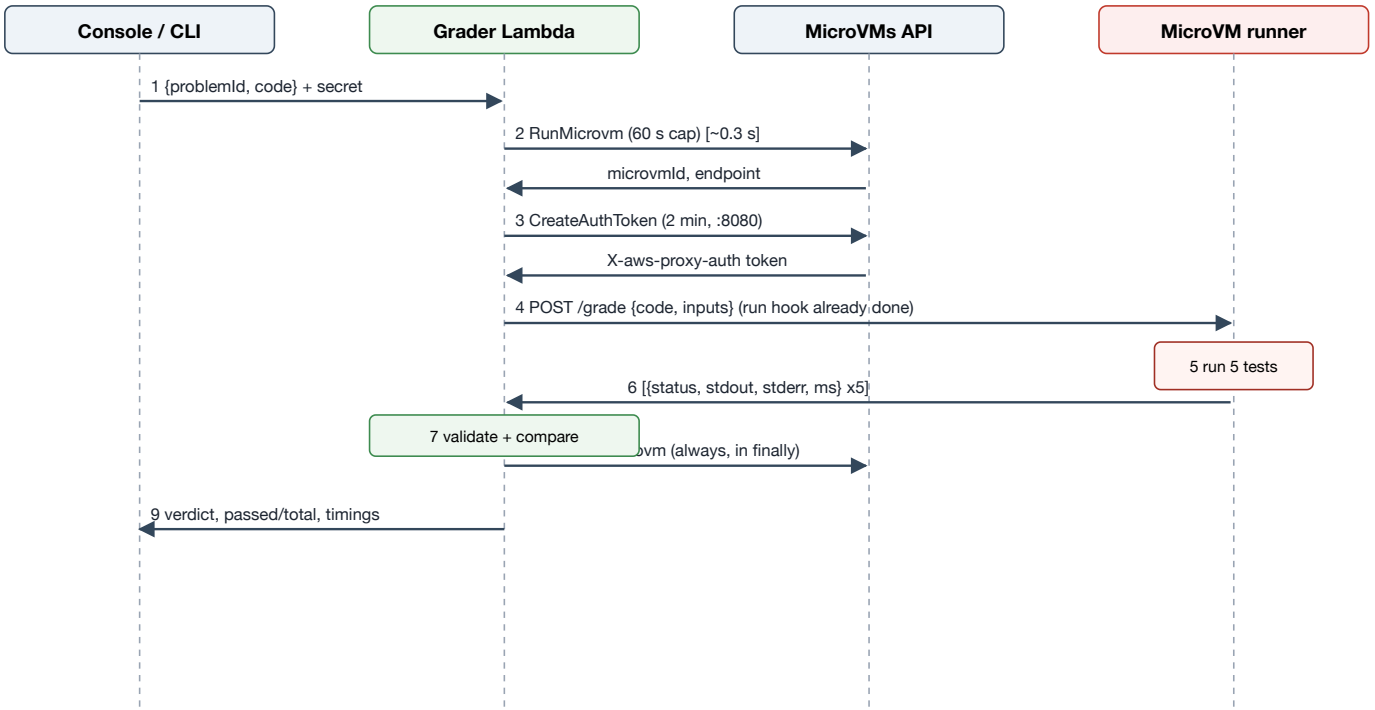
Resource	Name
MicroVM image	`grader-image` v4.0
S3 bucket	`mvm-grader-498556006298-apse1` (code zip)
IAM role	`mvm-grader-build` (image build)
Lambda + Function URL	`MicrovmGraderStack-Grader…` (CDK)
CDK bootstrap	`CDKToolkit` stack

1. Architecture



> Red arrow = the only data path into the VM. Expected answers stay in the trusted zone, so even a fully compromised runner can only lie about its own output.

2. How one submission runs



Timing field	What it measures	Typical (real run)
`launchMs`	RunMicrovm: restore snapshot	240 - 500 ms
`tokenMs`	CreateMicrovmAuthToken	about 190 ms
`gradeMs`	POST /grade incl. 5 sandboxed runs (infinite loop = 5 x 2 s)	3 - 4 s normal, about 13 s for infinite loop
`terminateMs`	TerminateMicrovm	about 160 ms

> Retries: RunMicrovm and the token call use exponential backoff with full jitter on `ThrottlingException`. The grader also retries the first POST briefly, because traffic only starts after the `run` hook returns.

3. Defence in depth: what stops each attack

Payload	What it tries	What stops it	Verdict
`infinite_loop.py`	burn CPU forever	`spawnSync` timeout 2000 ms + SIGKILL	TIMEOUT
`fork_bomb.py`	fork until the host dies	`prlimit --nproc=64` (fork refused)	ERROR
`memory_bomb.py`	allocate 100 GB	`prlimit --as=512 MB`	ERROR
`rm_rf.py`	delete /app, /usr, /etc, /bin	uid 1001, root-owned files: permission denied	ERROR
`exfil.py`	HTTPS to example.com	`unshare --net` : empty network namespace ("Name or service not known")	ERROR
`read_answers.py`	search disk for expected answers	answers never sent into the VM	WRONG_ANSWER
`kill_runner.py`	SIGKILL the runner and PID 1	different user: operation not permitted	ERROR

Layers

- **MicroVM boundary**: separate VM per submission, destroyed after.
- **No execution role**: nothing to steal.
- **In-VM sandbox**: user, network, process and memory limits.
- **Trusted comparison**: answers stay outside.
- **Hostile-reply handling**: size cap, zod schema, control characters stripped.
- **Cost cap**: 60 s max lifetime even if the grader crashes.

Real-service findings

- An empty `egressNetworkConnectors` list still attaches `INTERNET_EGRESS`. The first image let `exfil.py` reach example.com (HTTP 200). The network is therefore cut inside the VM.
- No "no egress" managed connector name exists (every guess was rejected).
- Lifecycle hooks are opt-in when creating the image.
- First request after restore is slow, so the `run` hook warms python up.
- The Lambda role needs `lambda:PassNetworkConnector` for the default connectors.
- Base image has Node 18 / Python 3.9 only, so the runner is bundled for node18.

4. Code map and demo flow

Path	Role
`runner/`	server inside the VM, sandbox, Dockerfile, esbuild bundle
`grader/`	grade(), MicroVM calls, retry, zod, compare, Lambda handler
`problems/two-sum/`	5 tests with expected outputs (trusted only)
`solutions/`, `payloads/`	correct, wrong, and 7 attack programs
`cli/`	submit, attack, parallel, cleanup, local:attack, events
`infra/`	CDK stack: Lambda, Function URL, IAM
`scripts/`, `iam/`	package, upload, create image; policies
`grader/test/`	15 vitest unit tests

Recording flow (console)

1. Show image `grader-image` (v4.0, state UPDATED, build logs).
2. Lambda function, Test tab: run the 9 events from `out/events/` in order.
3. Expected: ACCEPTED, WRONG_ANSWER (3/5), TIMEOUT, ERROR x4, WRONG_ANSWER, then ACCEPTED again.
4. Show the MicroVM list: every test was a separate VM, all TERMINATED.
5. Show CloudWatch: one `run-hook` line per VM, plus the Lambda `verdict` lines.

Cost and cleanup

- Each VM lives about 5 s, hard cap 60 s, always terminated.
- Teardown commands are in `CONSOLE-GUIDE.md` section E.